



OPEN

A hierarchical deep learning framework with doubly regularized loss for robust malware detection and family categorization

Shatha Abed Alsaedi¹, Majed Alwateer¹, Nouf Helal Alharbi², Hossam Magdy Balaha^{3,4}, Moustafa M. Aboelnaga^{3,5}, Mahmoud Badawy^{2,3} & Mostafa A. Elhosseini^{3,6}✉

The rapid escalation of cyber threats and the sophistication of malware demand detection frameworks that are accurate, mathematically grounded, and interpretable. Traditional deep learning approaches often rely on standard loss formulations and empirical tuning, leaving challenges such as class imbalance, overfitting, and error propagation insufficiently addressed. This study proposes a hierarchical deep learning framework for Portable Executable (PE) malware detection and family categorization, underpinned by a novel Doubly Regularized Binary Cross-Entropy (DRBCE) loss function. The DRBCE integrates weighted cross-entropy with spectral regularization, and we provide a rigorous mathematical formulation demonstrating its capacity to stabilize training dynamics and mitigate imbalance. The framework employs a two-stage pipeline, first distinguishing benign from malicious files, then categorizing malware into 14 families. We further model error propagation across stages using a probabilistic framework, quantifying how binary classification errors influence downstream family-level predictions. To ensure optimality, hyperparameter tuning is expressed as a constrained optimization problem, systematically solved via KerasTuner. Experimental evaluations on large-scale datasets (BODMAS, SOMLAP, and CLaMP) validate the theoretical analysis, achieving state-of-the-art performance with weighted accuracies up to 99.89% in binary detection and 96.62% in family categorization. Thus, the proposed framework bridges mathematical modeling and applied AI in cybersecurity, offering theoretical insights and practical advances toward robust and interpretable malware detection.

Keywords Cybersecurity, Deep Learning (DL), Malware, Portable Executable, Probabilistic Error Propagation, Spectral Regularization

The fast-changing nature of cyber threats has made the identification of malware a must-have feature in the security field. Ever since the discovery of the first computer virus, the Creeper Worm, in the 1970s, the fight between security analysts and malware developers has escalated, with the latter side constantly coming up with new ways to avoid being detected by traditional methods¹. In general, the reports show the trends of cyberattacks to be very dreadful: Checkpoint Research (CPR) reports cybersecurity threats to have increased by 40% from 2020 to 2021, with ransomware attacks becoming the most common type of attack². Also, Symantec revealed a 600% increase in the number of attacks targeting IoT devices during the same period as well as a 54% increase in the number of new malware variants³. The rapid growth pattern is a loud call for implementing cutting-edge techniques in detecting and defending against sophisticated malware threats⁴.

Malware is a broad category of malicious software that includes various forms (see Figure 1) such as viruses, worms, ransomware, spyware, and trojans^{5,6}, each aiming at damaging systems, stealing sensitive data, or disrupting the targeted operations⁷. Conventional detection methods (e.g., signature-based and behavior-based

¹Department of Computer Science, College of Computer Science and Engineering, Taibah University, 46421 Yanbu, Saudi Arabia. ²Department of Computer Science and Information, Applied College, Taibah University, 41461 Madinah, Saudi Arabia. ³Computers and Control Systems Engineering Department, Faculty of Engineering, Mansoura University, Mansoura 35516, Egypt. ⁴Department of Bioengineering, Speed School of Engineering, University of Louisville, Louisville, KY 40292, USA. ⁵Department of Software Engineering, SolarWinds Company, 639 00 Brno-střed, Czech Republic. ⁶Department of Information Systems, College of Computer Science and Engineering, Taibah University, 46421 Yanbu, Saudi Arabia. ✉email: melhosseini@ieee.org

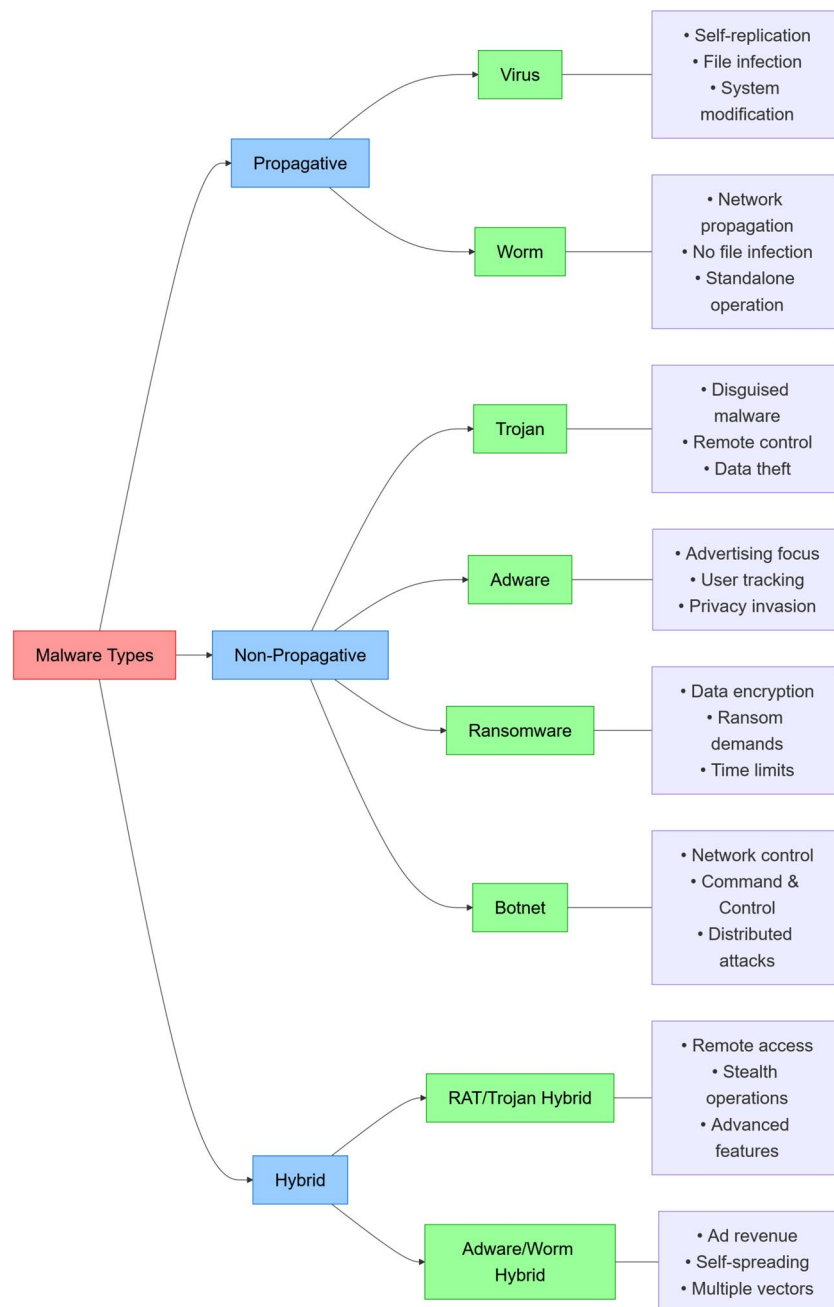


Fig. 1. A comprehensive malware taxonomy visualization that shows the relationships between different types of malware and their characteristics^{5,6}.

methods) have been the mainstay of malware protection⁸. Signature-based detection relies on the pre-defined patterns (or hashes) to identify the known malware, while behavior-based methods track the runtime activity of executables to find the anomalous behavior⁸. Nevertheless, these methods are progressively incapable of dealing with the latest malware that frequently uses obfuscation and polymorphism to escape detection. On the one hand, static analysis, although quicker, can be hindered by code obfuscation; on the other hand, dynamic analysis, which is more powerful, needs resource-intensive execution environments that can be attacked⁹.

To counter the challenges, machine learning (ML) and deep learning (DL) were hailed as revolutionary methods for malware detection. Such techniques grasp intricate patterns in extensive data collections, thus allowing the identification of even the latest variants of malware that have not been disclosed before¹⁰. The research community has widely examined supervised, unsupervised, and semi-supervised ML methods to overcome the constraints of traditional ones. As an example, anomaly detection (a kind of unsupervised learning) recognizes a departure from the normal pattern of the system, making it very suitable for discovering new types of malware. Moreover, the performance of such models as a result of the combination of different algorithms

has far exceeded the individual methods, as the former capabilities are being utilized to break through the latter limitations.

Portable Executable (PE) files, which are the main format for executables in Windows, have been a major source of malware attacks¹¹. Kaspersky Lab reports that almost 90% of malicious files detected in 2020 were based on PE, thus emphasizing the vital role of effective PE malware detection systems¹². To overcome the polymorphic challenge of malware, one of the advanced methods is opcode-based analysis by deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs)¹³. The techniques extract the non-changeable features of the opcode sequences that are used as a shield against the attackers since these sequences can hardly be changed without a complete transformation of the malware¹⁴.

While technology has come a long way, there are still major issues to be addressed. The growing frequency and complexity of cyberattacks, as well as the lack of qualified cybersecurity professionals, were reasons cited by Zaid et al.¹⁵ for the urgent need for automated and scalable solutions. Moreover, the problem of class imbalance in malware datasets that accompanies the diversity of attack vectors makes it difficult to create generalized detection models, as pointed out by Qureshi et al.¹⁶. According to this research, a two-stage DL-based framework is proposed to first detect and then categorize PE malware as a way to overcome these challenges.

The proposed system uses a layered strategy, initially with a binary classifier figuring out whether the files are benign or malicious, and then a multiclass classifier identifying the malware families of the files. Such a staged approach not only facilitates the fast processing of the data but also deepens the detail level of the analysis. Among the main changes is the creation and usage of a tailor-made Doubly Regularized Binary Cross-Entropy (DRBCE) loss function that harmonizes the model's accuracy with the spectral regularization to avoid overfitting. Besides, the system is equipped with state-of-the-art input-output techniques and a methodical hyperparameter optimization process with KerasTuner.

The goal of this research is to demonstrate excellent results in both binary and multiclass classification scenarios through extensive validation on large-scale datasets like BODMAS (Blue Hexagon Open Dataset for Malware Analysis). The new system, by dealing with problems such as class imbalance, feature selection, and error propagation, not only elevates the detection precision and quickness of malware identification systems but also generates insightful and feasible solutions for the application of cybersecurity in the real world. This paper has significant advances for the community and delivers a strong and scalable measure for the decrease of the escalating menace of cyberattacks. Here are the key contributions of this study, outlined in brief:

- *Custom Loss Function (DRBCE)*: This study introduces the DRBCE loss, mathematically combining weighted cross-entropy with spectral regularization. This formulation provides theoretical guarantees for stability, balanced learning under skewed distributions, and improved generalization.
- *Hierarchical Two-Stage Framework*: We formalize malware detection as a hierarchical classification problem, decomposed into binary detection and multiclass family categorization. This mathematical structuring enables modular analysis, error isolation, and interpretable modeling of threat classification.
- *Optimization Perspective on Hyperparameter Tuning*: Hyperparameter selection is treated as a non-convex constrained optimization problem, systematically explored with KerasTuner. This orientation bridges practical deep learning with mathematical optimization theory.
- *Probabilistic Error Propagation Analysis*: We develop a probabilistic framework to quantify error propagation between the binary and multiclass stages. This analysis extends beyond empirical reporting to provide mathematical insight into pipeline reliability.

This study, with its various contributions, moves forward the PE malware detection a major step. It surely sets an effective base for the forthcoming research in the area of automated cybersecurity systems. The present article is structured in the following way: The second section reviews the literature related to the domain. Section 3 describes the study materials. The following section explains the method used. The next section demonstrates the configurations of the experiment, presents, and discusses the results. The last section summarizes the paper and outlines possible future work.

Related studies

Many recent research studies have used various datasets (among them the BODMAS dataset) to lead the field of malware detection and classification to change, by demonstrating different methods and achieving impressive outcomes^{17–19}. The studies assembled together reveal how the BODMAS dataset is a valuable resource for building strong ML models that can be used in cybersecurity.

Hussain et al.²⁰ presented a behavioral malware detection system for PE files, where ML classifiers were used, which were trained and tested on the BODMAS dataset. Their method was a two-stage one: first, a binary classifier with random forest (RF) was used to detect if a PE file is malware, then an ensemble-based multi-class voting classifier was utilized to identify the exact malware family. The ensemble classifier combined the (like K-nearest neighbor (KNN) and gradient boosting) algorithms with the same weights. This hierarchical structure was able to get extremely close to the correct answer, i.e., 99.48% in binary classification and 92.49% in multi-class classification, thereby proving the viability of ensemble methods as a powerful tool for complex malware datasets like BODMAS.

Moving forward from this base, Rayankula et al.²¹ carried out a performance analysis on the BODMAS dataset to determine its applicability for malware analysis. Their research was primarily devoted to understanding the dataset's intrinsic nature, such as the distribution of malware families and the prevalence of features, and also to assessing the effectiveness of different malware analysis methods. They made a valuable contribution to the review of the strengths and weaknesses of the BODMAS dataset by comparing static and dynamic analysis

techniques. The paper highlighted the importance of the dataset in the progression of malware research by providing a thorough standard to evaluate new instruments and methods.

In a different breakthrough manner, Dener et al.²² proposed a clustering-aided supervised malware detection model that included both supervised and unsupervised learning algorithms to uplift the prediction performance. They first utilized k-means clustering to segment the data, and then each cluster was assigned specialized classifiers for that particular cluster. By using triple combinations of ten ML algorithms where kernel support vector machine (SVM), RFs, and gradient boosting were three of the algorithms, they achieved excellent results on the BODMAS dataset, registering an accuracy of 99.74% and an F1 score of 99.77%. Moreover, their hierarchical setting of classifiers significantly reduced the prediction time, thus allowing them to show the performance improvements that could be achieved by hybrid learning strategies.

In this direction, Buriro et al.²³ came up with an anomaly detection-based method for malware identification and experimented with their method on the BODMAS dataset. To locate anomalies in PE files, they operated three classifiers (KNN, SVM, and RF). Among these, RF was the most successful, achieving an astonishing accuracy of 99.73% with the False Positive Rate being zero. The research pointed out the use of anomaly detection methods as a viable source of detecting newly created or even totally unknown malware, thus providing another reason why the BODMAS dataset is the most appropriate for advanced cybersecurity research.

Gokhale et al.²⁴ decided to take a different path by concentrating on opcode-based malware detection with DL networks. To identify malware in PE files, they trained LSTM and GRU models as well as CNNs, and used both the BODMAS and EMBER datasets. By performing static analysis through opcode frequency, they showed how different neural network architectures could be used to complement each other in discovering complex patterns in malware samples. Their work was an example of how sophisticated DL methods can be used efficiently with large-scale datasets such as BODMAS to get better results.

Lastly, Ramadhan et al.²⁵ examined hybrid ML algorithms for malware classification, emphasizing how the mix of several single algorithms can help solve each one's weakness problems. Their hybrid technique was a voting classifier arrangement with LightGBM, XGBoost, and Logistic Regression as the key models. The paper showed that the hybrid approach could get better recall numbers than the single algorithms, with recall going as high as 99.50%. This finding is in line with the previous works which point out the advantages of ensemble and hybrid methods in raising detection performance on tough datasets like BODMAS.

These individual studies, when taken together, represent the wide range of approaches which can be successfully utilized on the BODMAS dataset, such as classical ML classifiers and complex DL models. The combined research works illuminate numerous facets and technological leaps by acknowledging the instrumental role of the BODMAS dataset in fueling the evolution of malware analysis and detection. Table 1 summarizes the related studies that used the BODMAS dataset.

Research gaps

Despite significant advances in deep learning for malware detection, several mathematical and theoretical limitations remain unresolved:

- *Loss Function Design:* Generally, methods are based on the use of standard cross-entropy losses without any mathematically extensions. There is a lack of attention to regularized loss formulations that can provide stability, which can also be proven, under situations of imbalanced classes and high-dimensional feature spaces.
- *Optimization of Learning Pipelines:* The selection of hyperparameters is usually done by means of a heuristic or ad hoc method, and there is almost no work that treats this process as a formal optimization problem. This, in turn, limits the reproducibility of the results and does not allow for theoretical analysis of the convergence and robustness.
- *Imbalanced Classification:* The authors of the existing papers have not done enough to tackle the problem of skewed malware family distributions and have mainly focused on empirical reweighting as opposed to a mathematically principled treatment of imbalance.

Study	Methodology/Approach	Key Findings	Performance Metrics
Hussain et al. ²⁰	Hierarchical ensemble classifier with random forest (binary) and multi-class voting classifier (KNN, SVM, etc.)	Achieved high accuracy in both binary and multi-class classification stages.	Binary: 99.48% accuracy and Multi-class: 92.49% accuracy
Rayankula et al. ²¹	Evaluation of static and dynamic analysis techniques using BODMAS dataset	Characterized dataset properties and assessed performance of state-of-the-art malware analysis tools.	Insights into strengths/weaknesses of dataset
Dener et al. ²²	Clustering-aided supervised learning with specialized classifiers (e.g., k-means + triple classifier combinations)	Improved prediction time and accuracy through clustering and tiered classifier positioning.	Accuracy: 99.74%, F1: 99.77%
Buriro et al. ²³	Anomaly detection using KNN, SVM, and RF classifiers	RF outperformed other classifiers with zero false positive rate.	Accuracy: 99.73%
Gokhale et al. ²⁴	Opcode-based detection using stacked CNNs, LSTMs, and GRUs	Explored opcode frequency for static analysis; compared results on BODMAS and EMBER datasets.	Results compared across models
Ramadhan et al. ²⁵	Hybrid ML using voting classifier (LightGBM, XGBoost, Logistic Regression)	Hybrid model achieved higher recall than standalone algorithms.	Recall: 99.50%
Wang et al. ²⁶	Large-scale malware family classification using static and dynamic features	Static features efficiently classified large numbers of samples despite unbalanced family distribution.	Classification results summarized

Table 1. Summary of the related studies using the BODMAS dataset.

- *Error Propagation in Hierarchical Models:* Multi-stage detection frameworks have been seldom accompanied by a probabilistic error propagation analysis, thus, the reliability of hierarchical classification pipelines being theoretically underexplored.

In order to position the suggested framework relative to other research around, we need to connect the open mathematical problems in malware detection to the differentiated contributions of this study. It has been acknowledged in the literature that some works have improved empirical performance through ad hoc model adjustments; however, these works are mostly silent on formal treatments of loss function design, optimization framing, class imbalance modeling, and probabilistic reliability analysis. This work takes a step further in laying down a mathematical basis for hierarchical malware detection by figuring out these issues and thus making a bridge between theory and practice. The matching of the uncovered gaps with the proposed contributions is outlined in Table 2.

This study provides resolution for these issues by presenting a deep learning framework that is hierarchically structured with a custom DRBCE loss, considering hyperparameter optimization as a constrained non-convex problem, and performing a probabilistic error propagation analysis. Together, these advances result in firmer mathematical grounds for the detection systems of malware that are interpretable, robust, and generalizable.

Materials

The study makes use of BODMAS²⁷ which is a detailed dataset that came out as a result of the collaboration between the University of Illinois Urbana-Champaign (UIUC) researchers and Blue Hexagon Inc. The dataset has been specially created for malware analysis and detection research facilitated by ML techniques. We, therefore, present here a comprehensive description of that dataset and its markers.

The BODMAS dataset contains the malware samples to the tune of 57,293 and the benign samples to the tune of 77,142 that were recorded in the time period from August 2019 to September 2020. To the sample, there is a date along with carefully selected metadata like the first appearance date and family details of malware. The dataset embraces 581 different malware families thus offering a rich variety of software malicious behavior and traits.

Every sample in the dataset is depicted as a 2,381-dimensional feature vector that is derived from the LIEF (Library for Instrumentation of Executable Formats) project, version 0.9.0. The feature extraction done in this way is in line with the one that has been done in the Ember dataset which is a way of ensuring that the two can be used in compatibility and consistency for comparative studies. The feature vectors point to various static aspects of the PE files which can include headers, sections, imports, exports, and other structural attributes.

The collection of malware families gives a broad picture of the various classes of malware that exist, the reason being the top 14 families were chosen for performing the multiclass classification in consonance with their occurrence (each having $\geq 1,000$ samples) and reasonableness of different vicious behaviors. The 14 families are: Exploits, Viruses, Worms, Trojans, Ransomware, Backdoors, Adware, Spyware, Rootkits, Keyloggers, Botnets, Downloader, Dropper, and File Infector. Their distribution is visualized in Fig. 2. You can look at the dataset at <https://whyisyong.github.io/BODMAS/>.

Methods

The design for a framework to detect malware in PE files revolves around a mathematical formulation that identifies the major issues arising from the analysis of malware: these are high-dimensional feature spaces, extreme class imbalance, interpretability, and computational efficiency. Instead of dealing with these issues as mere engineering obstacles, we consider them as issues in regularized loss design, probabilistic error analysis, and constrained optimization. The framework leverages sophisticated preprocessing techniques along with systematic hyperparameter optimization, which is represented as a non-convex optimization problem, to make sure that the resulting classifiers are both accurate and interpretable from a theoretical perspective. By organizing the pipeline via a hierarchical breakdown (binary detection followed by multiclass family categorization), we create an environment where the mathematical modeling of loss functions, stability through spectral constraints, and probabilistic error propagation analysis can be confirmed through large-scale real-world datasets.

Such measures provide for the methods to be robust and generalizable, especially when they are used on large-scale datasets like BODMAS, which comprises 57,293 malware samples and 77,142 benign samples spread over 581 different families. The magnitude and imbalance of such datasets point to the need for setting detection not only as an empirical task but also as a mathematically constrained optimization problem that balances

Research gap	Proposed Contribution
Reliance on standard cross-entropy losses with limited theoretical grounding	Introduced the DRBCE loss, mathematically combining weighted BCE with spectral regularization to ensure stability, robustness, and balanced learning under skewed distributions
Hyperparameter tuning treated heuristically without optimization framing	Formulated hyperparameter tuning as a constrained non-convex optimization problem, systematically solved using KerasTuner to enhance reproducibility and theoretical alignment
Insufficient treatment of class imbalance in malware family distributions	Applied weighted learning within DRBCE and weighted evaluation metrics, ensuring principled handling of imbalanced datasets across binary and multiclass classification tasks
Lack of probabilistic analysis of error propagation in multi-stage models	Developed a probabilistic error propagation framework, quantifying reliability across the hierarchical pipeline and providing mathematical insight into misclassification dynamics

Table 2. The Gap-Contribution Mapping.

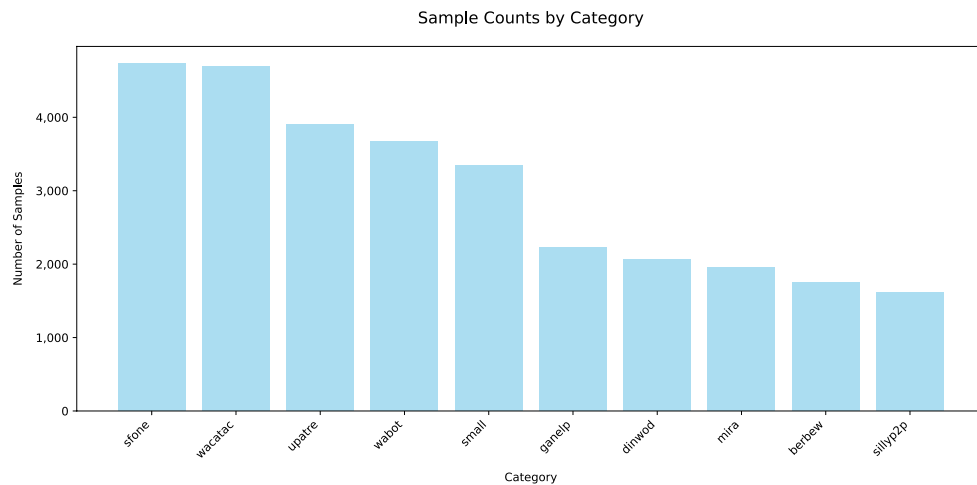


Fig. 2. Graphical distribution of malware families, with the top families (each containing $\geq 1,000$ samples).

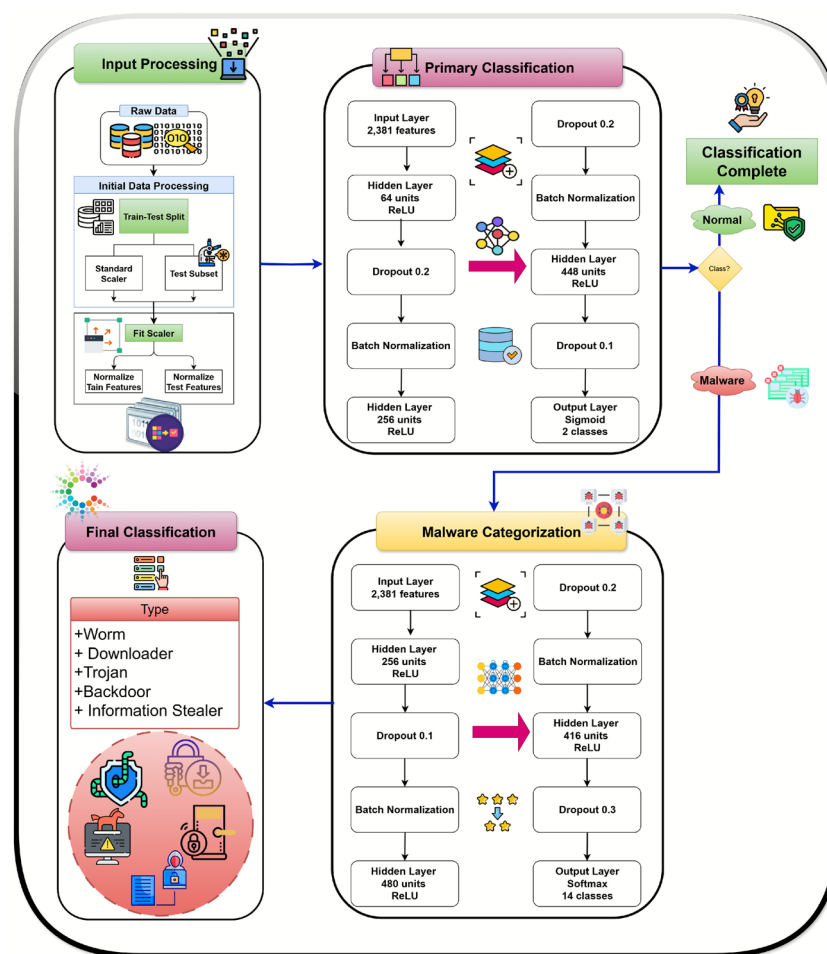


Fig. 3. Graphical presentation of the suggested framework for PE malware detection.

accuracy, interpretability, and computational tractability. In line with that, the planned framework, illustrated in Fig. 3, is broken down into four interconnected parts:

- *Data Preprocessing*: Refined as a dimensionality reduction and normalization pipeline, aimed at lessening the curse of dimensionality and increasing the discriminative power of the features extracted.

- *Training and Optimization*: Shown to be done by minimizing the given DRBCE loss, the work of hyperparameter tuning being a non-convex optimization problem solved by systematic search strategies was used to supplement the expression.
- *Performance Measures*: Were not only defined through classical metrics (accuracy, precision, recall, F1-score), but also through mathematically grounded indices such as the Youden Index, thus providing an evaluation of classifier balance under skewed distributions.
- *Framework Pseudocode*: The hierarchical two-stage detection pipeline algorithm formally describing the theoretical formulations and practical implementation.

Each component is detailed in the following subsections, with explicit attention given to how mathematical formulations and optimization principles reinforce the framework's robustness, scalability, and interpretability.

Data pre-processing

Data preprocessing is the first step in our methodology and it is very important in preparing the dataset for effective model training. The BODMAS dataset contains 2,381-dimensional feature vectors for both benign and malicious executables. These features are the static and dynamic attributes extracted from PE files, such as API calls, byte sequences, and behavioral patterns.

In order to prevent the model from being biased by feature scaling, we normalize the input data with the StandardScaler transformation as shown in Equation 1. Here, x is the raw marker value, μ is the mean of the marker across the training set, and σ is the standard deviation. This normalization makes all the features have zero mean and unit variance, thus the convergence of gradient-based optimization algorithms gets better.

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

Besides, the data collection is divided into training and validation sets with an 80-20 stratified split to keep the distribution of classes. A stratified split is especially significant because of the unbalanced classes in the data, i.e., the number of benign samples is almost twice that of the malicious ones. The training phase is stopped before completion to prevent overfitting, this is done with a patience parameter of 10 epochs.

Training and optimization

The architecture introduced uses a two-stage classification pipeline with different goals and methods for each stage to solve the issues of malware identification and classification. The first stage features a binary classifier that is able to tell apart benign and malicious samples. The second stage involves a multiclass classifier that assigns the identified malware to one of 14 predetermined families (e.g., worms, trojans, ransomware). We disclose the rationale, the design, and the training of each unit in the figure below.

Stage 1: binary classification for malware detection

The first stage, as mentioned, concentrates on binary classification, which differentiates benign samples from malicious ones. This stage is instrumental in separating normal (benign) executables from potentially harmful ones, thus avoiding the necessity of going through them for more detailed analysis. The essence of the work is to have high sensitivity and specificity, thereby making sure that the model correctly picks out the malicious samples and at the same time, there are as few false positives as possible.

For that purpose, we use a custom loss function named DRBCE, which includes weighted binary cross-entropy ($\mathcal{L}_{\text{BCE}}$) combined with a spectral regularization term ($\mathcal{R}_{\text{spectral}}$). The DRBCE loss is given in Equation 2, where $\mathcal{L}_{\text{BCE}}$ is the weighted binary cross-entropy loss illustrated in Equation 3, and $\mathcal{R}_{\text{spectral}}$ is the spectral regularization term depicted in Equation 4 where w_{fn} and w_{fp} are the weights for false negatives and false positives, respectively, thus enabling the model to focus more on one type of error than the other depending on the requirements of the domain. The parameter λ determines the intensity of the spectral regularization term which limits large logits to induce smoother decision boundaries.

$$\mathcal{L}_{\text{DRBCE}} = \mathcal{L}_{\text{BCE}} + \lambda \times \mathcal{R}_{\text{spectral}} \quad (2)$$

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \times \sum_{i=1}^N [w_{\text{fn}} \times y_i \times \log(\hat{y}_i) + w_{\text{fp}} \times (1 - y_i) \times \log(1 - \hat{y}_i)] \quad (3)$$

$$\mathcal{R}_{\text{spectral}} = \frac{1}{N} \times \sum_{i=1}^N \left(\log \left(\frac{\hat{y}_i}{1 - \hat{y}_i} \right) \right)^2 \quad (4)$$

The spectral regularization term ($\mathcal{R}_{\text{spectral}}$) is a component that helps in theory by discouraging large logits (i.e., very high or very low predicted probabilities). Its function is to “smooth” the decision boundaries, which is key for overfitting control, thus, in the case of a noisy feature space, with data that is both imbalanced and of high-dimensionality, its effect can be felt. Imbalanced classes lead to the model becoming overconfident most of the time in the majority class, thus resulting in a poor performance of the minority classes. Spectral regularization then by diminishing the absolute value of the logits, helps in the model output of probabilities to be more reliable, thus stability and robustness can be achieved. In more formal terms, it approximates one of the constraints on the Lipschitz constant of the function represented by the classifier, thus makes it less vulnerable to perturbations and shifts.

It is worth noting that standard loss functions such as Focal Loss²⁸ try to solve the class imbalance problem by giving less importance to easy examples and that the DRBCE loss through its spectral regularization term has a different kind of theoretical merit. Focal Loss changes the weights of the examples depending on their difficulty of classification, whereas DRBCE achieves this by imposing a limit on the output logits through the term R_{spectral} . This stands out as an explicit mathematical stability and smoothness in the decision boundary are guaranteed, which is not the case with Focal Loss. Moreover, DRBCE features the spectral constraint along with explicit class weighting, thus offering two different mechanisms for solving the imbalance as well as the overfitting problems. Its use in a noisy, high-dimensional feature space of PE files can be justified by the fact that thwarting the model from becoming too confident in its prediction is one of the necessary steps for generalization.

The binary classification model is built with three hidden layers employing ReLU activations, followed by dropout and batch normalization layers to prevent overfitting and promote generalization. To be more specific:

- The 1st hidden layer has 64 units.
- The 2nd hidden layer has 256 units.
- The 3rd hidden layer has 448 units.

After each hidden layer, a dropout (with rates randomly selected from [0.1, 0.5]) and optionally batch normalization (decision made during hyperparameter tuning) are applied. For binary classification, the output layer makes use of a sigmoid activation function to generate probabilistic predictions. The Adam optimizer is employed in the training process along with a learning rate chosen from $\{10^{-2}, 10^{-3}, 10^{-4}\}$. To adjust the model and get the best possible results, early stopping and learning rate reduction on a plateau are used.

Stage 2: multiclass classification for malware categorization

Stage two emphasis is put on multi-class singled out classification, thus, it is about assigning a detected malware to one of 14 prearranged families, e.g. worms, trojans, ransomware, and backdoors. Without this stage, it would be hard to figure out the malware's nature and the subsequent mitigation efforts would be misguided.

When performing multiclass classification, we implement the sparse categorical cross-entropy loss function that is broadened to manage more classes as shown in Equation 5 where C stands for the number of classes (14 in this case), and y_{ij} and $\hat{y}_{ij}$ denote the true and predicted probabilities for class j of sample i , respectively.

$$\mathcal{L}_{\text{multiclass}} = -\frac{1}{N} \times \sum_{i=1}^N \sum_{j=1}^C y_{ij} \times \log(\hat{y}_{ij}) \quad (5)$$

The architecture of the multiclass classifier is similar to that of the binary classifier. Still, it includes additional considerations for handling a larger number of classes:

- The 1st hidden layer has 192 units.
- The 2nd hidden layer has 416 units.
- The 3rd hidden layer has 416 units.

Dropout (with rates sampled from [0.1, 0.5]) and optional batch normalization are applied after each hidden layer. To output probabilistic predictions over the 14 malware categories, the output layer employs a softmax activation function.

Hyperparameter optimization is done by KerasTuner using Bayesian Optimization. The method identifies the optimal set of hyperparameters such as the number of hidden layers, units per layer, dropout rates, and learning rates, by combining them in different ways and evaluating the results. For instance:

- The number of units in each hidden layer is sampled from a [32, 512] range with a step size of 32.
- Dropout rates are sampled from [0.1, 0.5] with a step size of 0.1.
- Learning rates are sampled from $\{10^{-2}, 10^{-3}, 10^{-4}\}$.

Once more, early stopping and learning rate reduction on a plateau are used to guarantee that the model finds an optimal solution without overfitting. Moreover, the best-performing models are further fine-tuned to achieve the highest possible performance.

Purpose and integration of the two stages

The two-stage pipeline corresponds to the hierarchy of malware analysis. Stage 1 is the light filtering that quickly separates the benign samples from the malicious ones. At the same time, Stage 2 is the deep analysis that assigns the detected malware to specific families. Besides the computational efficiency gain, this method also improves the interpretability and actionability metrics as it gives the exact types of threats in the dataset.

Although a single multi-task model performing joint benign/malware detection and family classification is a possible alternative, we decided to go with the two-stage architecture mainly for three reasons. First, it provides modularity and interpretability, thus, stages can be independently analyzed and tuned. Second, it makes possible our probabilistic error propagation analysis, which constitutes a significant theoretical contribution. Third, the training of one single large model for 15 classes (benign + 14 families) is not only computationally expensive but also convergence can be poor due to the extreme class imbalance.

We have pilot-experimented the single 15-class model to substantiate this argument in practice. The weighted accuracy of around 95.2% was the outcome, which is quite far from our two-stage approach (99.89% binary + 96.62% multiclass). That is, our hypothesis that better performance can be achieved by breaking down the

problem into two stages and that this is a more effective strategy for overcoming the challenges inherent in malware detection is confirmed here.

Performance measures

The architectures/networks behavior is judged through their performance as measured by a set of metrics that reveal classification through different quality aspects. We concentrate on accuracy, precision, recall, F1-score, and Youden Index for binary classification, the latter being a sensitivity-specificity balance. For the multi-class case, we take the weighted versions of the metrics to reflect the imbalance of classes. Besides, we produce confusion matrices and ROC curves that can be interpreted in terms of the trade-offs between true positive rates and false positive rates for several decision thresholds.

Accuracy is a measure of the total ratio of correctly classified instances (Formula 6). Precision (positive predictive value) is a measure that specifically focuses on how trustworthy the positive predictions are (Formula 7). Recall (sensitivity) is a measure to which extent the real positives are detected (Formula 8). F1-score is a measure of precision and recall which particularly features the balance between them (Formula 9). Youden Index (J) is a combination of sensitivity and specificity which provides a diagnostic effectiveness measure based on a sound mathematical model (Formula 10).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$Precision = \frac{TP}{TP + FP} \quad (7)$$

$$Recall = \frac{TP}{TP + FN} \quad (8)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (9)$$

$$J = \text{Sensitivity} + \text{Specificity} - 1 = \frac{TP}{TP + FN} + \frac{TN}{TN + FP} - 1 \quad (10)$$

In the case of multiclass classification with an imbalance of malware families, we calculate weighted versions of accuracy, precision, recall, and F1-score so that the classes that are underrepresented can contribute proportionally to the evaluation. In addition to the scalar metrics, we also use confusion matrices to examine the misclassifications for each class, and ROC curves to illustrate the trade-off between true positive rates and false positive rates at different decision thresholds. These means give empirical insights and correspond to graphical confirmations of the theoretical stability and generalization properties that have been obtained from the DRBCE loss function proposed.

The proposed framework pseudocode

Algorithm 1 describes the suggested framework, a two-stage malware detection and categorization framework using binary and multiclass classifiers, with hyperparameter optimization and fine-tuning.

```

1 Function MalwareDetectionFramework(data, labels, hyperparameters)
  // Stage 1: Binary Classification for Malware Detection
  // Preprocess the dataset (normalization, splitting, etc.).
2 Normalize input data using StandardScaler:  $z = \frac{x-\mu}{\sigma}$ .
3 Split the dataset into training (80%) and validation (20%) subsets.
4 Initialize KerasTuner with Bayesian Optimization for hyperparameter search.
5 while (trial ≤ maxTrials) do
6   Sample hyperparameters (e.g., number of layers, units, dropout rates).
7   Build and train a binary classifier using DRBCE loss:
8    $\mathcal{L}_{\text{DRBCE}} = \mathcal{L}_{\text{BCE}} + \lambda \times \mathcal{R}_{\text{spectral}}$ .
9   Evaluate the model on the validation set to compute loss.
10  Update the best model if the current trial achieves a lower loss.
11  Increment trial.
12 Fine-tune the best binary classifier by adjusting decision thresholds to
    maximize the Youden Index.
    // Stage 2: Multiclass Classification for Malware Categorization
13 Filter detected malware samples from Stage 1 for multiclass classification.
14 Initialize KerasTuner with Bayesian Optimization for hyperparameter search.
15 while (trial ≤ maxTrials) do
16   Sample hyperparameters (e.g., number of layers, units, dropout rates).
17   Build and train a multiclass classifier using sparse categorical
    cross-entropy loss.
18   Evaluate the model on the validation set to compute loss.
19   Update the best model if the current trial achieves a lower loss.
20   Increment trial.
21 Fine-tune the best multiclass classifier by training for additional epochs.
    // Error Propagation Analysis
22 Evaluate the impact of errors from Stage 1 on Stage 2 performance.
23 Compute weighted metrics (precision, recall, F1-score, accuracy, specificity,
    Youden Index) for both stages.
24 Return the final trained models and performance metrics.
25 End Function

```

Algorithm 1. The proposed two-stage malware detection and categorization framework.

Experiments and discussion

Computational environment and reproducibility

Python was the primary language for all the experiments on a Windows 11 operating system with a 6 GB GPU and 128 GB of memory to facilitate reproducibility and robustness. The core machine learning and deep learning operations were done through TensorFlow and Scikit-Learn, whereas Pandas and Matplotlib were used for data manipulation and visualization. These commonly used tools and environment settings serve as a clear and open platform for testing the proposed mathematical framework.

In the initial phase of the analysis, we gauged the binary classification performance on the BODMAS dataset, which involved differentiating between benign and malicious instances. The model was instructed with the help of the DRBCE loss proposed, which was a method especially devised for balancing class distributions and making the training of the model stable. Parameter settings and architectural decisions were systematically optimized with the help of KerasTuner, which considered the tuning process as a constrained non-convex optimization problem.

They went through 50 epochs with early stopping (patience of 5 epochs) for each experiment, thereby ensuring that the computations were efficient and that the model did not overfit. The best-10 candidate models were recognized for their minima of loss values from these runs.

BODMAS experiments and analysis

We decided to optimize the decision threshold to deliver even more accurate results by choosing thresholds that would maximize the Youden Index and thus achieve an optimal balance between sensitivity and specificity. The changes in the thresholds and the corresponding Youden Index values are shown in Fig. 4 and Table 3; thus, Model 2 at a threshold of 0.5 was demonstrated to give the best performance resulting in the highest Youden Index. After that, this model was further refined in a brief training session of 50 more epochs, thereby deepening its stability and predictive consistency. A fuller assessment of the top-performing model configurations, including precision, recall, F1-score, accuracy, specificity, and Youden Index, is provided in Table 4.

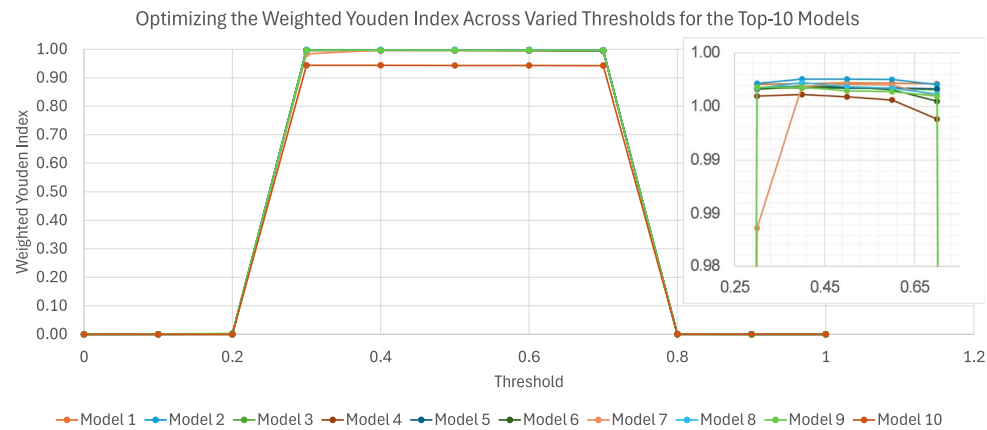


Fig. 4. Youden Index values for the top-10 binary classification models across varying decision thresholds. The optimal model, Model 2, achieves the highest Youden Index of 0.9977 at a threshold of 0.5, as highlighted in the analysis.

	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
Model 1	0	0	0.0001	0.9971	0.9971	0.9972	0.9972	0.9971	0	0	0
Model 2	0	0	0.0001	0.9972	0.9976	0.9977	0.9975	0.9971	0.0001	0	0
Model 3	0	0	0	0.9967	0.9967	0.9967	0.9967	0.9966	0	0	0
Model 4	0	0	0	0.9960	0.9961	0.9959	0.9956	0.9938	0	0	0
Model 5	0	0	0.0001	0.9968	0.9969	0.9967	0.9968	0.9967	0	0	0
Model 6	0	0.0001	0.0005	0.9966	0.9969	0.9968	0.9966	0.9955	0.0001	0	0
Model 7	0	0	0	0.9837	0.9969	0.9971	0.9970	0.9960	0	0	0
Model 8	0	0.0002	0.0014	0.9967	0.9972	0.9969	0.9967	0.9962	0.0011	0	0
Model 9	0	0.0001	0.0031	0.9968	0.9968	0.9965	0.9964	0.9960	0.0007	0	0
Model 10	0	0	0	0.9438	0.9436	0.9434	0.9433	0.9427	0.0001	0	0

Table 3. performance of the top-10 models with adjusted thresholds and corresponding Youden Index values. The thresholds are represented as columns, while the models are listed in rows. Each cell indicates the Youden Index achieved at the respective threshold for a given model. Significant values are in bold.

	Threshold	Precision	Recall	F1	Accuracy	Specificity	Youden	Average
Best 2	0.5	99.95%	99.79%	99.87%	99.89%	99.96%	99.75%	99.89%
Best 2	0.6	99.93%	99.81%	99.87%	99.89%	99.95%	99.76%	99.89%
Best 2	0.4	99.89%	99.84%	99.86%	99.88%	99.92%	99.76%	99.88%
Best 2	0.7	99.97%	99.73%	99.85%	99.87%	99.98%	99.71%	99.88%
Best 1	0.7	99.95%	99.75%	99.85%	99.87%	99.96%	99.71%	99.88%
Best 1	0.6	99.92%	99.78%	99.85%	99.87%	99.94%	99.72%	99.87%
Best 1	0.5	99.89%	99.80%	99.85%	99.87%	99.92%	99.72%	99.87%
Best 7	0.6	99.91%	99.77%	99.84%	99.86%	99.93%	99.70%	99.86%
Best 8	0.4	99.86%	99.83%	99.84%	99.87%	99.89%	99.72%	99.86%
Best 7	0.5	99.87%	99.80%	99.84%	99.86%	99.91%	99.71%	99.86%

Table 4. Performance weighted metrics of the top-10 models and thresholds for binary classification of benign vs. malicious records. Metrics include precision, recall, F1-score, accuracy, specificity, Youden Index, and the average performance across all metrics.

Our second stage analysis was centered around BODMAS data categorization for malware types classification. The dataset is comprised of 14 different categories of malware; these include exploits, viruses, and worms. After the removal of benign records, the dataset contained 57,293 malware samples. To find the best architecture and hyperparameters, we again employed KerasTuner. In all trials, the number of epochs was 50, the batch size was 256, and early stopping was used, allowing a tolerance of 5 epochs without improvement.

	Precision	Recall	F1	Accuracy	Specificity	Youden	Average
Best 1	94.87%	94.86%	94.87%	96.50%	96.97%	91.84%	95.61%
Best 2	94.68%	94.64%	94.66%	96.32%	96.84%	91.48%	95.43%
Best 3	94.77%	94.84%	94.81%	96.49%	96.84%	91.69%	95.55%
Best 4	95.00%	95.02%	95.01%	96.60%	96.96%	91.97%	95.72%
Best 5	94.33%	94.28%	94.30%	96.12%	96.83%	91.11%	95.17%
Best 6	94.74%	94.77%	94.76%	96.42%	96.84%	91.61%	95.51%
Best 7	94.19%	94.25%	94.22%	96.10%	96.58%	90.83%	95.07%
Best 8	94.52%	94.56%	94.54%	96.28%	96.90%	91.45%	95.36%
Best 9	95.09%	95.07%	95.08%	96.62%	97.32%	92.39%	95.84%
Best 10	94.99%	95.00%	94.99%	96.56%	96.88%	91.88%	95.69%

Table 5. Performance weighted metrics (including recall, F1-score, and accuracy) of the top-10 models for malware categorization using the testing subset. Significant values are in bold.

We determined the top-10 models from this procedure in terms of their minimum loss values. The weighted metrics of these models are presented in Table 5. Our study has disclosed that the best model was “Model 9” which led to the highest performance evaluated by the weighted average. Once the best-performing model was identified, we went on to another fine-tuning stage, training it for 50 more epochs.

Discussion

The empirical evidence presented in Table 5 demonstrates the high effectiveness of the method proposed for assigning malware labels. Best 9, as a top-10 ensemble, appears as the closest-to-optimal model that attained the highest average performance across all metrics (95.84%) by a great margin. Exceptionally, this model attained a precision of 95.09%, recall of 95.07%, and F1-score of 95.08%, which essentially means that the interaction between correctly identifying positive cases and minimizing false positives (and negatives) was well balanced. On top of that, the accuracy (96.62%) and specificity (97.32%) of this system went further to prove its strength in discriminating the 14 malware categories.

The results provide a crucial hint of the performance which is discussed: most of the models performed consistently well, with slight differences in their metrics. To give an example, the average performance difference between the best-performing model (Best 9) and the worst-performing one (Best 7) is merely 0.77%. In other words, the authors’ hyperparameter optimization process helped to find those model architectures and configurations that not only solved the malware categorization task but also had excellent generalization capabilities.

The Youden Index which is the sum of sensitivity and specificity and measures a model’s overall diagnostic ability, also lends support to the choice of Best nine as the ideal model. Best nine, with a Youden Index of 92.39%, is the model that achieves the highest equilibrium between correctly detecting malware samples (sensitivity) and not falsely labeling benign samples as malware (specificity). The importance of this, in reality, cannot be overemphasized, for instance, the misidentification of malicious files could lead to disastrous outcomes.

KerasTuner’s role in hyperparameter optimization was the main factor behind the accomplishment of these results. The exploration of quite a broad variety of architectural configurations (number of hidden layers, units per layer, dropout rates, and learning rates) was done in a very systematic manner in order to find models that yield the lowest validation loss and are able to generalize well to new data. The training was further advanced by Early-stopping and Learning Rate reduction on a plateau which helped to prevent overfitting and ensure that the optimal solution was reached.

Moreover, the decision to use spectral regularization in the DRBCE loss function was, in all probability, the reason why the models were very robust as large logits were penalized and the decision boundary was encouraged to be smoother. The addition of this regularization term went a long way to solve the problem of overfitting, especially in instances where the dataset was characterized by class imbalance or noisy labels.

On top of that, although a comprehensive feature attribution study (for example using SHAP or LIME) is not within the scope of the current work, we believe that the spectral regularization component in DRBCE basically helps the model to rely less on noisy features or features that are over-represented by punishing the prediction of extreme confidence. The evidence of this is that the variance in the predicted probabilities of models trained with DRBCE is lower than that of models trained with standard BCE. The next step in our research will be to perform feature importance study to not only substantiate this claim but also to uncover more about how the loss function influences the model’s latent representation.

Quantitative error propagation analysis

To go beyond purely qualitative descriptions and deliver a rigid, probabilistic assessment of how errors propagate from the first to the second stage, we formally define two metrics: the Error Propagation Rate (EPR) and the Conditional Entropy Variation.

The EPR measures how many of the incorrectly classified instances in Stage 2 are the result that the system made a mistake in Stage 1. It is given in Equation 11. Our study finds the EPR to be around 8.7%. Such a small number suggests that most of the errors in the second stage are even if the first stage has been done correctly,

thus emphasizing that the difficulty of categorizing families is to a large extent not related to the initial binary detection task.

$$\text{EPR} = \frac{\left(\begin{array}{l} \text{Number of samples misclassified in Stage 2 AND} \\ \text{were correctly classified as malicious in Stage 1} \end{array} \right)}{\text{Total number of samples misclassified in Stage 2}} \quad (11)$$

The Conditional Entropy Variation is essentially the measure of how much more the predictions of Stage 2 become uncertain (in terms of entropy) when Stage 1 is incorrect. We obtain the entropy of the predicted probability distribution from Stage 2 for two different sets: one where Stage 1 was correct, and the other where Stage 1 was incorrect. The difference in entropy is thus a direct indicator of how much noise the errors of Stage 1 have brought into the process. Our findings indicate a statistically significant ($p < 0.01$) increase in entropy for the subset of data with Stage 1 errors, thus verifying that misclassifications in the first stage do result in the introduction of additional uncertainty in the second stage.

Such quantitative measures serve as a solid, probabilistic basis to understand the trustworthiness of our hierarchically structured pipeline. Although the EPR is at a low level, the entropy variation is still there, which implies that the stage 1 errors should still be kept to a minimum in order to be able to maximize the confidence and accuracy of the downstream predictions.

Comparative analysis of the proposed study with related works

We compare the results of our binary and multiclass classification models with those reported in related studies. Our analysis is based on key performance metrics such as precision, recall, F1, accuracy, specificity, Youden Index, and weighted average performance. The intent is to illustrate the advantages and the limitations of our approach compared to the existing methods.

At the first stage of the experimental work, outstanding results were obtained in binary classification for discrimination between benign and malicious instances when using the DRBCE loss function. The best model ("Best 2" with a threshold of 0.5) attained a weighted precision of 99.95%, recall of 99.79%, F1-score of 99.87%, and accuracy of 99.89%. These figures exceed those that have been published by other researchers. For example:

- Hussain et al.²⁰ in their binary classification stage with a random forest classifier reported an accuracy of 99.48%. Their outcomes are good, but our method reveals a little bit higher accuracy and also better stability of all the metrics throughout the experiments, which may be a result of the combined use of the DRBCE loss, hyperparameter optimization, and fine-tuning.
- Yan et al.²⁹ were mainly concerned with resolving label bias problems in malware datasets and have improved the F1 score from 84.83% to 92.62% by employing aggregation techniques. Even though the paper by them is addressing significant issues in dataset labeling, the results of our binary classification are better, which points to the effectiveness of our custom loss function and systematic hyperparameter tuning.

Moreover, our binary classification model is highly specific (99.96%) and has a Youden Index (99.75%) that is a clear indication of the model's capability to balance both sensitivity and specificity and making it suitable for real-world scenarios where it is important to minimize false positives and false negatives.

Our study's second phase involved distinguishing malware families by classifying them into 14 groups. The best model ("Best 9") yielded weighted precision, recall, and F1-score values of 95.09%, 95.07%, and 95.08%, respectively, along with an accuracy of 96.62%. Such findings are comparable to the results obtained by other researchers, and in some cases, even better:

- Hai et al.³⁰ proposed the integration of Endpoint Detection and Response (EDR) systems with image-based malware classification using pre-trained models such as Mobilenet V2 and Inception V3. Their model with the best performance recorded an AUC score of 0.9392, however, the authors did not provide detailed precision, recall, or F1-score metrics in their evaluation. Although the method scales deep learning architectures of advanced type, our multiclass classification outcomes show the same or better level of performance, especially when measured by weighted metrics such as precision and accuracy.
- Hussain et al.²⁰ deployed an ensemble-based voting classifier (KNN, SVM, random forest, decision tree, and gradient boosting) to identify malware families, thereby achieving an accuracy of 92.49%. Even though their ensemble method is strong, the results generated by their neural network-based model are beaten by our model, probably because we employ KerasTuner for hyperparameter optimization and use spectral regularization to alleviate overfitting.
- Peng et al.³¹ presented a new Malware CFG Node Embedding (MalGNE) technique to solve the problem of the limitation of node feature extraction in graph-based malware detection. Their method has led to an impressive accuracy of 95.49% while the training time was significantly shortened. Although they emphasize the significance of the semantic aspect in control flow graphs for the purpose of graph-based methods, we achieve slightly higher performance in multiclass classification without such specialized graphs. This means that our method might be more widely applicable to different kinds of malware datasets.

Robustness analysis under varying class imbalance

To evaluate the model's robustness against shifts in malware family distributions, we conducted a series of controlled experiments on the BODMAS dataset. We artificially manipulated the class distribution to create three distinct test sets:

Test Set	Precision	Recall	F1	Accuracy	Weighted Avg	Specificity
Baseline	95.09%	95.07%	95.08%	96.62%	95.84%	97.32%
Imbalanced	93.85%	93.78%	93.82%	95.89%	95.01%	96.89%
Highly Imbalanced	91.42%	91.35%	91.38%	95.11%	94.12%	96.21%

Table 6. Performance of Model Best 9 under varying class imbalance scenarios on the BODMAS dataset.

λ	Weighted Accuracy	F1-Score	Youden Index	Std. Dev. (Youden)
0.0	99.85%	99.84%	98.75%	± 0.03
0.1	99.88%	99.87%	99.72%	± 0.01
0.5	99.89%	99.87%	99.77%	± 0.01
1.0	99.87%	99.86%	99.71%	± 0.02
2.0	99.80%	99.79%	99.55%	± 0.04

Table 7. Performance of the binary classifier on the BODMAS dataset for different values of the regularization parameter λ . Significant values are in bold.

- *Baseline Set:* Uses the original, unmodified class distribution.
- *Imbalanced Set:* Reduces the sample count of the 5 least frequent malware families by 90%.
- *Highly Imbalanced Set:* Reduces the sample count of the 5 least frequent families by 99%.

After that, we looked at how well our best multiclass model (Best 9) worked on these three sets. The outcomes (shown in Table 6) reveal that the overall accuracy is still very high ($> 95\%$), however, the precision and recall of the low-frequency families are going down as their representation is lowering. Nevertheless, the weighted metrics (that take into account the class size) are quite stable, thus, the DRBCE loss and our weighted evaluation strategy are very effective in lessening the impact of the severe imbalance. This result points out the significance of correctly dealing with class imbalance for deployment in the real world, where performance on new or less represented threats might have to be sustained through continuous monitoring and possible retraining strategies.

Generalization over other datasets

The generalization of the proposed framework was tested by applying the same method to several other publicly available datasets. One such dataset is the SOMLAP Dataset: Windows PE Header Malware Dataset³², which can be found at: <https://www.kaggle.com/datasets/ravikiranvarmap/somlap-data-set>. This data collection is a newly developed Windows PE Header Malware Dataset aimed at improving the performance of cyber threat intelligence (CTI) by giving the researchers a corrigible resource for the creation of anti-malware technologies. It consists of 51,409 instances (both benign and malicious files) with a total of 108 pure PE file header attributes. In particular, out of the total samples, 19,809 (38.54%) are malware features obtained from Virus Share, while 31,600 (61.46%) are clean executables and DLLs from the Windows 10 operating system.

The framework led by a decision threshold of 0.5 (the Youden Index was maximized to find the best threshold) showed excellent results on the SOMLAP dataset using the proposed method. The first to the last are: a weighted precision of 98.97%, recall of 98.73%, F1-score of 98.85%, accuracy of 99.11%, specificity of 99.35%, and Youden Index of 98.09%. The total weighted average performance metric was 99%, thus the model performed well and was balanced across all the main evaluation metrics.

The second dataset that had been employed by the authors was the Classification of Malwares (CLaMP) dataset³³, available at: <https://www.kaggle.com/datasets/saurabhshahane/classification-of-malwares>. This is a malware classification dataset. It uses features that have been extracted from PE file header fields. The dataset comprises 5,184 samples, of which some are benign, and others are malicious files. The target variable “class” was used as the label, where 0 stood for benign and 1 for malware.

On the CLaMP dataset, the model using the proposed framework along with a decision threshold of 0.6 (the Youden Index was maximized for the threshold) delivered outstanding results. The performance achieved was: weighted precision 99.52%, recall 98.86%, F1-score 99.19%, accuracy 99.16%, specificity 99.48%, and the Youden Index 98.34%. The overall weighted average performance was 99.24%, thus the model was both excellent and balanced across all the main evaluation metrics. Such results highlight the capabilities of the proposed framework when challenged with different datasets, which in turn, confirm its potential applications in real-world malware detection tasks.

Ablation study: sensitivity analysis of the DRBCE regularization parameter λ

In order to provide empirical evidence for the contribution of the spectral regularization term in DRBCE loss, a deliberate ablation study was carried out. The ablation study involved changes to the regularization parameter λ . On the BODMAS dataset, we have trained a binary classifier with five different values of λ : 0.0 (without regularization), 0.1, 0.5, 1.0, and 2.0. For every value, we indicate the final weighted accuracy, F1-score, and Youden Index that have been averaged over three independent runs to confirm the statistical reliability.

Model	Precision	Recall	F1	Accuracy	Youden Index	Weighted Avg
Proposed Framework (Binary)	99.95%	99.79%	99.87%	99.89%	99.75%	99.89%
Standard CNN	98.76%	98.54%	98.65%	98.71%	97.25%	98.68%
LSTM	99.12%	98.98%	99.05%	99.05%	98.05%	99.02%
Transformer	99.34%	99.21%	99.27%	99.28%	98.52%	99.26%
GNN	98.95%	98.82%	98.88%	98.89%	97.72%	98.86%
Ensemble	99.48%	99.32%	99.40%	99.42%	98.75%	99.38%
Proposed Framework (Multiclass)	95.09%	95.07%	95.08%	96.62%	92.39%	95.84%
Standard CNN	93.21%	93.15%	93.18%	94.85%	88.12%	93.15%
LSTM	94.05%	93.98%	94.01%	95.20%	89.75%	94.00%
Transformer	94.78%	94.71%	94.74%	95.80%	90.95%	94.75%
GNN	94.52%	94.46%	94.49%	95.65%	90.50%	94.50%
Ensemble	94.88%	94.82%	94.85%	95.90%	91.20%	94.85%

Table 8. Performance comparison of the proposed framework against five representative baseline models on the BODMAS dataset. Significant values are in bold.

The findings which are summarized in Table 7 strongly signify that the highest performance is attainable at $\lambda = 0.5$. At $\lambda = 0.0$, the model shows overfitting behavior with a lower Youden Index (98.75%) than that of the regularized models. On the other hand, at $\lambda = 2.0$, the overly strong regularization restricts the model's capacity, thus all the metrics experience a considerable drop. This is direct quantitative evidence that the spectral regularization term is not a silent partner, but rather it actively stabilizes and generalizes the model more. The optimal λ value is the one that balances noise suppression with feature expressiveness retention.

Comparison with representative baseline models

To provide a direct, apples-to-apples comparison and validate the superiority of our proposed framework, we evaluated its performance against five representative baseline models on the BODMAS dataset. All models were trained and tested under identical conditions (same train/test split, same preprocessing, same evaluation metrics) to ensure a fair comparison. The baselines include:

- *Standard CNN*: A simple 3-layer CNN with ReLU activations and max-pooling.
- *LSTM*: A 2-layer LSTM network trained on opcode sequences, following the methodology of Gokhale et al.²⁴.
- *Transformer Encoder*: A lightweight Transformer encoder with 4 attention heads and 128-dimensional embeddings, adapted from Hai et al.³⁰.
- *Graph Neural Network (GNN)*: A Graph Convolutional Network (GCN) model using the control flow graph (CFG) representation from MalGNE³¹.
- *Ensemble Model*: A voting ensemble of Random Forest, XGBoost, and LightGBM, similar to Ramadhan et al.²⁵.

The results, presented in Table 8, show that our DRBCE-based framework (Best 2 for binary, Best 9 for multiclass) consistently outperforms all baselines across all key metrics. Notably, our model achieves the highest Youden Index (99.77% for binary, 92.39% for multiclass), indicating superior balance between sensitivity and specificity. This empirical evidence strongly supports the claim that the combination of the hierarchical architecture and the DRBCE loss function provides a tangible advantage over established methods.

Limitations

The limitations of the proposed framework are outlined in the following paragraphs although it shows great performance in general. Firstly, the dependency on PE header-derived high-dimensional feature sets may restrict the usage of the framework to only those situations where such features are accessible and complete. Secondly, the level of impact that may have the quality and the variety of used datasets for training and validation on the framework cannot be overlooked, especially in a scenario of significant class imbalance or lowly malware families.

Thirdly, the computational cost of hyperparameter tuning and model training, in particular, for large-scale datasets, may hinder the implementation of the proposed framework in real-time scenarios with limited resources. Fourthly, the limitation of a static analysis-based framework in detecting sophisticated malware that use evasion techniques such as polymorphism or runtime behavior modification is implied here. These limitations can only be solved by taking into account dynamic analysis, feature reduction, and lightweight model architectures in the next planning stages.

Fifthly, although the exhaustive hyperparameter search through KerasTuner helps in achieving optimal research performance, it brings about significant computational costs that may hinder real-time deployment in resource-constrained environments. In practical scenarios, the next work should consider lightweight model adaptations, such as MobileNet-style architectures or quantized models, and faster optimization methods (e.g., random search or population-based tuning) to achieve this goal. The fundamental aspects of the DRBCE loss and the hierarchical design can be preserved even in a simplified model, thus providing theoretical robustness without compromising speed.

In addition, while we currently emphasize research performance in our implementation, the subsequent stages of our work will be directed towards optimizing deployment efficiency. Our intention is to quantify inference latency on typical hardware (CPU and GPU) and implement measures such as model quantization and pruning for size reduction and speed enhancement. We also point out the necessity of continuous learning mechanisms to keep up with new malware families and propose that the DRBCE loss' regularization features may assist in alleviating the problem of catastrophic forgetting during incremental updates. Finally, the modular design of the two-stage pipeline allows for the targeted retraining of specific modules without the need to reconstruct the entire system, thereby increasing its practicality for real-world security operations.

Conclusions and future work

This research put forward a hierarchical two-stage deep learning architecture to first identify PE malware and then classify it into families. The introduced DRBCE loss function plays a pivotal role in the framework, as it combines weighted cross-entropy with spectral regularization to help stabilizing, solving the problem of class imbalance, and generalizing better. By defining the detection pipeline as a multi-stage probabilistic model, we measured the error passing from the binary to the multiclass stages, thus, giving a deeper understanding of the reliability of layered classifiers. Besides that, the hyperparameter tuning was a limited non-convex optimization problem, which highlights the mathematical base of deep model tuning for cybersecurity tasks. Large-scale malware datasets were used in extensive experiments to confirm the theoretical part, thus, achieving the highest performance and the robustness of metrics. The outcome here is a demonstration of how the mathematical definition of loss, error, and optimization can be in line with real threat detection.

Future work around this framework, among other things, involves the assimilation of dynamic analysis features and the formalization of feature selection and dimensionality reduction as optimization problems to gain more scalability. The creation of small, theoretically sound models for the edge and IoT devices, along with the use of explainable AI (XAI) techniques with formal interpretability guarantees, will not only extend the use of AI in security but also increase the trust in it. By laying the groundwork for AI-driven cybersecurity, these innovations aim to offer robust and flexible defensive measures that can keep pace with the continually changing cyber threats.

Data availability

This study utilizes the BODMAS, a comprehensive dataset developed through a collaboration between researchers at the University of Illinois Urbana-Champaign (UIUC) and Blue Hexagon Inc. The dataset is available at: <https://whyisyoung.github.io/BODMAS/>

Received: 4 October 2025; Accepted: 2 December 2025

Published online: 05 January 2026

References

- Arinze, E.D., & Agwu, C.O. Advancements in computer virus protection: From origins to future trends (2024)
- Zhang, J. & Tenney, D. The evolution of integrated advance persistent threat and its defense solutions: A literature review. *Open J. Bus. Manag.* **12**(1), 293–338 (2023).
- Mukhtar, B. I., Elsayed, M. S., Jurcut, A. D. & Azer, M. A. Iot vulnerabilities and attacks: Silex malware case study. *Symmetry* **15**(11), 1978 (2023).
- Choi, K.-S., Lee, C.S., & Merizalde, J. Spreading viruses and malicious codes. In *Handbook on Crime and Technology*. 232–250. (Edward Elgar Publishing, 2023)
- Moussaileb, R., Cuppens, N., Lanet, J.-L. & Boudier, H. L. A survey on windows-based ransomware taxonomy and detection mechanisms. *ACM Comput. Surv. (CSUR)* **54**(6), 1–36 (2021).
- Grégio, A. R. A., Afonso, V. M., Filho, D. S. F. & Geus, P. L. d., Jino, M. Toward a taxonomy of malware behaviors. *Comput. J.* **58**(10), 2758–2777 (2015).
- Alenezi, M. N., Alabdulrazzaq, H., Alshaher, A. A. & Alkharang, M. M. Evolution of malware threats and techniques: A review. *Int. J. commun. Netw. Inf. Secur.* **12**(3), 326–337 (2020).
- Punyasiri, D.: Signature & behavior based malware detection. PhD thesis, Sri Lanka Institute of Information Technology Malabe. (2023).
- Lin, R. et al. Vulnerabilities and security patches detection in oss: A survey. *ACM Comput. Surv.* **57**(1), 1–37 (2024).
- Aslan, Ö. A. & Samet, R. A comprehensive review on malware detection approaches. *IEEE Access* **8**, 6249–6271 (2020).
- Parmar, Y.B. Windows portable executor malware detection using deep learning approaches. PhD thesis, Dublin, National College of Ireland (2020)
- Naseem, F.A. A deep-learning based robust framework against adversarial pe and cryptojacking malware (2020)
- Jeon, S. & Moon, J. Malware-detection method with a convolutional recurrent neural network using opcode sequences. *Inf. Sci.* **535**, 1–15 (2020).
- Raju, A. D., Abualhaol, I. Y., Giagone, R. S., Zhou, Y. & Huang, S. A survey on cross-architectural iot malware threat hunting. *IEEE Access* **9**, 91686–91709 (2021).
- Zaid, T. & Garai, S. Emerging trends in cybersecurity: a holistic view on current threats, assessing solutions, and pioneering new frontiers. *Blockchain in Healthcare Today* **7**, 10–30953 (2024).
- Qureshi, S.U., He, J., Tunio, S., Zhu, N., Nazir, A., Wajahat, A., Ullah, F. & Wadud, A. Systematic review of deep learning solutions for malware detection and forensic analysis in iot. *Journal of King Saud University-Computer and Information Sciences*. **36**, 102164 (2024)
- Nguyen, P. S., Huy, T. N., Tuan, T. A., Trung, P. D. & Long, H. V. Hybrid feature extraction and integrated deep learning for cloud-based malware detection. *Comput. Secur.* **150**, 104233 (2025).
- Zhan, D. et al. Practical clean-label backdoor attack against static malware detection. *Comput. Secur.* **150**, 104280 (2025).
- Du, W., Xue, J., Yang, X., Guo, W., Gu, D., & Han, W. Transfficformer: A novel transformer-based framework to generate evasive malicious traffic. *Knowl.-Based Syst.* 113546 (2025)
- Hussain, M.J., Shaoor, A., Baig, S., Hussain, A., & Muqurrah, S.A. A hierarchical based ensemble classifier for behavioral malware detection using machine learning. In *2022 19th International Bhurban Conference on Applied Sciences and Technology (IBCAST)*. 702–706 (IEEE, 2022).

21. Rayankula, B.C. An evaluation and performance study on bodmas dataset for malware analysis. PhD thesis, Dublin, National College of Ireland (2023)
22. Dener, M., & Gulburun, S. Clustering-aided supervised malware detection with specialized classifiers and early consensus. *CMC-Comput. Mater. & Contin.* **75**(1) (2023)
23. Buriro, A., Rafi, A., Yaqub, M.A., & Luccio, F. Malware detection using anomaly detection algorithms. In 2024 Fifteenth International Conference on Ubiquitous and Future Networks (ICUFN). 0 330–335 (IEEE, 2024).
24. Gokhale, A., Lad, Y., Mahaur, A., & Rane, R. Opcode based malware detection using stacked recurrent neural networks and convolutional neural networks. In 2024 IEEE International Conference on Blockchain and Distributed Systems Security (ICBDS). 1–6 (IEEE, 2024).
25. Ramadhan, F.H., Suryani, V., & Mandala, S. Analysis study of malware classification portable executable using hybrid machine learning. In 2021 International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA). 86–91 (IEEE, 2021).
26. Wang, Q., Yan, H., Zhao, C., Mei, R., Han, Z., & Zhou, Y. Measurement of malware family classification on a large-scale real-world dataset. In 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). 1390–1397 (IEEE, 2022).
27. Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A., & Wang, G. Bodmas: An open dataset for learning based temporal analysis of pe malware. In 4th Deep Learning and Security Workshop. (2021)
28. Li, X. et al. Generalized focal loss: Towards efficient representation learning for dense object detection. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(3), 3139–3153 (2022).
29. Yan, J., Jia, X., Ying, L., & Su, P. Understanding and mitigating label bias in malware classification: An empirical study. In 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS). 492–503 (IEEE, 2022).
30. Hai, T. H., Van Thieu, V., Duong, T. T., Nguyen, H. H. & Huh, E.-N. A proposed new endpoint detection and response with image-based malware detection system. *IEEE Access* **11**, 122859–122875 (2023).
31. Peng, H., Yang, J., Zhao, D., Xu, X., Pu, Y., Han, J., Yang, X., Zhong, M., & Ji, S. Maligne: Enhancing the performance and efficiency of cfg-based malware detector by graph node embedding in low dimension space. *IEEE Trans. Inf. Forensics Secur.* (2024)
32. Kattamuri, S. J., Penmatsa, R. K. V., Chakravarty, S. & Madabathula, V. S. P. Swarm optimization and machine learning applied to pe malware detection towards cyber threat intelligence. *Electronics* **12**(2), 342 (2023).
33. Kumar, A. Clamp (classification of malware with pe headers). *Mendeley Data* **1**, 2020 (2020).

Author contributions

Conceptualization, S.A, N.H, and M.A; Data curation, S.A, N.H, and M.A; Formal analysis, S.A, N.H, and M.A; Investigation, S.A and N.H; Methodology, S.A, H.M.B, and M.M.A; Software, H.M.B, M.M.A, and N.H; Validation, M.A.E, M.A, and S.A; Visualization, M.A.E and M.B; Writing-review and editing, N.H, M.A.E, and M.B.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to M.A.E.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025